

AI/ML LAB JOURNAL

Name: Yash Yogesh Mhatre

Roll: C25067

Div: B

MCAL22 Artificial Intelligence and Machine Learning Lab

INDEX

Sr. No	Artificial Intelligence Section	Date	CO	Sign
1.	Implementation of Logic programming using PROLOG DFS for water jug problem		CO1	
2.	Implementation of Logic programming using PROLOG BFS for tic-tac-toe problem		CO1	
3.	Implementation of Logic programming using PROLOG Hill-climbing to solve 8- Puzzle Problem.		CO1	
4.	Introduction to Python Programming: Learn the different libraries - NumPy, Pandas, SciPy, Matplotlib, Scikit Learn.		CO2	
5.	Implement Perceptron algorithm for OR operation		CO2	
6.	Improve the prediction accuracy by estimating the weight values for the training data using stochastic gradient descent. (Perceptron)		CO2	
7.	Implement Adaline algorithm for AND operation		CO2	

Sr. No	Machine Learning Section	Date	CO	Sign
1.	Implementation of Features Extraction and Selection, Normalization, Transformation, Principal Components Analysis.		CO3	
2.	Implementation of Logistic regression		CO4	
3.	Implementation of Classifying data using Support Vector Machine (SVM)- Linear and Non-Linear SVM Classification		CO4	
4.	Implement Elbow method for K means Clustering		CO4	
5.	Implementation of Bagging Algorithm: Random Forest		CO4	
6.	Implementation of Boosting Algorithms: AdaBoost, Stochastic Gradient Boosting, Voting Ensemble		CO4	

Artificial Intelligence Section

1. Implementation of Logic programming using PROLOG DFS for water jug problem

Code:

```
%Practical 1 on Water jug problem 20Apr26
```

```
%Define goal state
```

```
goal(state(2, _)). % 2 liters in jug A
```

```
%Define capacity
```

```
capacity(4, 3).
```

```
%Move rules
```

```
%fill jug A
```

```
move(state(X, Y), state(4, Y)) :-  
    X < 4.
```

```
%fill jug B
```

```
move(state(X, Y), state(X, 3)) :-  
    Y < 3.
```

```
%empty jug A
```

```
move(state(X, Y), state(0, Y)) :-  
    X > 0.
```

```
%empty jug B
```

```
move(state(X, Y), state(X, 0)) :-  
    Y > 0.
```

```
%pour A -> B
```

```
move(state(X, Y), state(X1, Y1)) :-  
    X > 0,  
    Y < 3,  
    Transfer is min(Y, 3 - Y),  
    X1 is X - Transfer,  
    Y1 is Y + Transfer.
```

```
%pour B -> A
```

```
move(state(X, Y), state(X1, Y1)) :-  
    Y > 0,  
    X < 4,  
    Transfer is min(Y, 4 - X),  
    Y1 is Y - Transfer,  
    X1 is X + Transfer.
```

```
% DFS algorithm
```

```
dfs(State, Visited, [State|Path]) :-
```

```
goal(State).
```

```
dfs(State, Visited, [State|Path]) :-  
    move(State, Next),  
    \+ member(Next, Visited),  
    dfs(Next, [Next|Visited], Path).
```

```
%Start predicate
```

To execute:

```
consult('D:/MCA/SEM_2/AI/AILPracticals/YYM_AI_WaterJug.pl').  
solve(Path) :-  
dfs(state(0, 0), [state(0, 0)], Path).
```

Output:

```
1 ?- consult('D:/MCA/SEM_2/AI/AILPracticals/YYM_AI_Pr1_WaterJug.pl').  
Warning: d:/mca/sem_2/ai/ailpracticals/yym_ai_pr1_waterjug.pl:43:  
Warning: Singleton variables: [Visited,Path]  
true.  
  
2 ?- solve(Path).  
Path = [state(0, 0), state(4, 0), state(4, 3), state(0, 3), state(3, 0), state(3, 3), state(4, 2), state(0, 2), state(, )|...] .  
  
3 ?- solve(Path).  
Path = [state(0, 0), state(4, 0), state(4, 3), state(0, 3), state(3, 0), state(3, 3), state(4, 2), state(0, 2), state(, )|...] ;  
Path = [state(0, 0), state(4, 0), state(4, 3), state(0, 3), state(3, 0), state(3, 3), state(4, 2), state(0, 2), state(, )|...] ;  
Path = [state(0, 0), state(4, 0), state(1, 3), state(4, 3), state(0, 3), state(3, 0), state(3, 3), state(4, 2), state(0, 2), state(, )|...] ;  
Path = [state(0, 0), state(4, 0), state(1, 3), state(4, 3), state(0, 3), state(3, 0), state(3, 3), state(4, 2), state(0, 2), state(, )|...] ;  
Path = [state(0, 0), state(4, 0), state(1, 3), state(0, 3), state(3, 0), state(3, 3), state(4, 2), state(0, 2), state(, )|...] ;  
Path = [state(0, 0), state(4, 0), state(1, 3), state(0, 3), state(3, 0), state(3, 3), state(4, 2), state(0, 2), state(, )|...] ;  
Path = [state(0, 0), state(4, 0), state(1, 3), state(1, 0), state(0, 1), state(4, 1), state(4, 3), state(0, 3), state(, )|...] ;  
Path = [state(0, 0), state(4, 0), state(1, 3), state(1, 0), state(0, 1), state(4, 1), state(2, 3), state(4, 3), state(, )|...] ;  
Path = [state(0, 0), state(4, 0), state(1, 3), state(1, 0), state(0, 1), state(4, 1), state(2, 3), state(0, 3), state(, )|...] ;  
Path = [state(0, 0), state(4, 0), state(1, 3), state(1, 0), state(0, 1), state(4, 1), state(2, 3), state(2, 0), state(, )|...] ;  
Path = [state(0, 0), state(4, 0), state(1, 3), state(1, 0), state(0, 1), state(0, 3), state(3, 0), state(3, 3), state(, )|...] ;  
Path = [state(0, 0), state(4, 0), state(1, 3), state(1, 0), state(0, 1), state(0, 3), state(3, 0), state(3, 3), state(, )|...] ;  
Path = [state(0, 0), state(0, 3), state(4, 3), state(4, 0), state(1, 3), state(1, 0), state(0, 1), state(4, 1), state(, )|...] ;  
Path = [state(0, 0), state(0, 3), state(3, 0), state(4, 0), state(1, 3), state(1, 0), state(0, 1), state(4, 1), state(, )|...] ;  
Path = [state(0, 0), state(0, 3), state(3, 0), state(4, 0), state(1, 3), state(1, 0), state(0, 1), state(4, 1), state(, )|...] ;  
Path = [state(0, 0), state(0, 3), state(3, 0), state(3, 3), state(4, 3), state(4, 0), state(1, 3), state(1, 0), state(, )|...] ;  
Path = [state(0, 0), state(0, 3), state(3, 0), state(3, 3), state(4, 3), state(4, 0), state(1, 3), state(1, 0), state(, )|...] ;  
Path = [state(0, 0), state(0, 3), state(3, 0), state(3, 3), state(4, 2), state(4, 3), state(4, 0), state(1, 3), state(, )|...] ;  
Path = [state(0, 0), state(0, 3), state(3, 0), state(3, 3), state(4, 2), state(4, 3), state(4, 0), state(1, 3), state(, )|...] ;  
Path = [state(0, 0), state(0, 3), state(3, 0), state(3, 3), state(4, 2), state(0, 2), state(2, 0), state(4, 0), state(, )|...] ;  
Path = [state(0, 0), state(0, 3), state(3, 0), state(3, 3), state(4, 2), state(0, 2), state(2, 0), state(2, 3), state(, )|...] ;  
Path = [state(0, 0), state(0, 3), state(3, 0), state(3, 3), state(4, 2), state(4, 0), state(1, 3), state(1, 0), state(, )|...] ;  
Path = [state(0, 0), state(0, 3), state(3, 0), state(3, 3), state(4, 2), state(4, 0), state(1, 3), state(1, 0), state(, )|...] ;  
false.  
  
3 ?- 
```

2. Implementation of Logic programming using PROLOG BFS for tic-tac-toe problem

Code:

```
% --- Initial Board -----
initial([e, e, e,
        e, e, e,
        e, e, e]).

% --- Goal -----
% X wins
goal(Board) :-
    win(Board, x).

% --- Winning Conditions -----
win([P, P, P, _, _, _, _, _], P). % row 1
win([_, _, _, P, P, P, _, _], P). % row 2
win([_, _, _, _, _, P, P, P], P). % row 3
win([P, _, _, P, _, _, P, _], P). % col 1
win([_, P, _, _, P, _, _, P], P). % col 2
win([_, _, P, _, _, P, _, _], P). % col 3
win([P, _, _, P, _, _, _], P). % diagonal
win([_, _, P, _, P, _, _], P). % anti-diagonal

% --- Player Management -----
next_player(x, o).
next_player(o, x).

% --- Move Generation -----
% Place Player's mark on any empty cell
move(Board, Player, NewBoard) :-
    nth0(Index, Board, e),
    replace(Board, Index, Player, NewBoard).

% --- List Utilities -----
% Replace element at Index with X
replace([_ | T], 0, X, [X | T]).
replace([H | T], I, X, [H | R]) :-
    I > 0,
    I1 is I - 1,
    replace(T, I1, X, R).

% --- BFS Implementation -----
% Base case: front of queue is a winning board
bfs([[Board | Path] | _], _, [Board | Path]) :-
    goal(Board),
    !.

% Recursive case: expand current board, append children to queue
bfs([[Board | Path] | Queue], Player, Solution) :-
    findall(
        [NextBoard, Board | Path],
        move(Board, Player, NextBoard),
        Children
```

```

    ),
    append(Queue, Children, NewQueue),
    next_player(Player, NextPlayer),
    bfs(NewQueue, NextPlayer, Solution).

% --- Top-Level Solver -----
solve(Path) :-
    initial(Board),
    bfs([[Board]], x, RevPath),
    reverse(RevPath, Path).

% --- Pretty Printing -----
print_board([A, B, C, D, E, F, G, H, I]) :-
    format("~w | ~w | ~w~n", [A, B, C]),
    format("----+----+----~n"),
    format("~w | ~w | ~w~n", [D, E, F]),
    format("----+----+----~n"),
    format("~w | ~w | ~w~n~n", [G, H, I]).

print_solution([]).
print_solution([Board | Rest]) :-
    print_board(Board),
    print_solution(Rest).

```

AIML Practical 17 Apr 2026 Tic Tac Toe BFS
 Software to execute: SWI Prolog
 consult('path with backward slash'). to set up
 solve(Path), print_solution(Path) to execute

Output:

```

23 ?- consult('D:/MCA/SEM_2/AI ML/AI MLPracticals/YVM_AI_Pr2_tictac.pl').
true.

24 ?- solve(Path), print_solution(Path).
  e | e | e
  ---+---+---
  e | e | e
  ---+---+---
  e | e | e

  e | x | e
  ---+---+---
  e | e | e
  ---+---+---
  e | e | e

  x | x | e
  ---+---+---
  e | e | e
  ---+---+---
  e | e | e

  x | x | x
  ---+---+---
  e | e | e
  ---+---+---
  e | e | e

Path = [[e, e, e, e, e, e, e, e|...], [e, x, e, e, e, e, e, e|...], [x, x, e, e, e, e, e, e|...], [x, x, x, e, e, e, e, e|...]].

25 ?- .

```

3. Implementation of Logic programming using PROLOG Hill-climbing to solve 8- Puzzle Problem.

Code:

```
% --- Goal State -----
goal([1, 2, 3,
      4, 5, 6,
      7, 8, 0]). % 0 represents blank

% --- Entry Point -----
solve(Start) :-
    hill_climb(Start, []).

% --- Hill Climbing Algorithm -----
% Base case: goal reached
hill_climb(State, _) :-
    goal(State),
    write('Goal Reached:'), nl,
    print_state(State).

% Recursive case: move to the best child
hill_climb(State, Visited) :-
    get_children(State, Children),
    exclude(member_of(Visited), Children, ValidChildren),
    evaluate_children(ValidChildren, ScoredChildren),
    sort(ScoredChildren, Sorted), % sort ascending by heuristic
    Sorted = [[_, BestChild] | _], % pick best (lowest score)
    write('Current State:'), nl,
    print_state(State), nl,
    hill_climb(BestChild, [State | Visited]).

% Stuck: no valid children to explore
hill_climb(State, _) :-
    write('Stuck at local optimum:'), nl,
    print_state(State).

% --- Membership Check -----
member_of(List, Element) :-
    member(Element, List).

% --- Child Generation -----
get_children(State, Children) :-
    findall(Child, move(State, Child), Children).

% --- Possible Moves -----
move(State, NewState) :- swap(State, 0, -3, NewState). % up
move(State, NewState) :- swap(State, 0, 3, NewState). % down
move(State, NewState) :- swap(State, 0, -1, NewState). % left
move(State, NewState) :- swap(State, 0, 1, NewState). % right

% --- Swap Positions -----
swap(State, Zero, Move, NewState) :-
    nth0(Index, State, Zero),
    NewIndex is Index + Move,
```



```

valid_move(Index, NewIndex),
nth0(NewIndex, State, Value),
set_element(State, Index, Value, Temp),
set_element(Temp, NewIndex, Zero, NewState).

% --- Valid Move Guard -----
valid_move(I, J) :-
    J >= 0, J < 9,
    \+ invalid_edge(I, J).

% Prevent wrap-around between rows
invalid_edge(2, 3).
invalid_edge(3, 2).
invalid_edge(5, 6).
invalid_edge(6, 5).

% --- List Element Replacement -----
set_element(_ | T, 0, X, [X | T]).
set_element([H | T], I, X, [H | R]) :-
    I > 0,
    I1 is I - 1,
    set_element(T, I1, X, R).

% --- Heuristic: Misplaced Tiles -----
heuristic(State, H) :-
    goal(Goal),
    count_misplaced(State, Goal, H).

count_misplaced([], [], 0).
count_misplaced([0 | T1], [_ | T2], H) :-    % skip blank tile
    count_misplaced(T1, T2, H).
count_misplaced([H | T1], [H | T2], C) :-    % tiles match
    count_misplaced(T1, T2, C).
count_misplaced([H1 | T1], [H2 | T2], C) :-    % tiles differ
    H1 \= H2,
    count_misplaced(T1, T2, C1),
    C is C1 + 1.

% --- Evaluate Children -----
evaluate_children([], []).
evaluate_children([State | Rest], [[H, State] | ScoredRest]) :-
    heuristic(State, H),
    evaluate_children(Rest, ScoredRest).

% --- Print State -----
print_state([A, B, C, D, E, F, G, H, I]) :-
    write([A, B, C]), nl,
    write([D, E, F]), nl,
    write([G, H, I]), nl.

```

Output:

```
27 ?- consult('D:/MCA/SEM_2/AI ML/AI MLPracticals/YM_AI_Pr3_HillClimb.pl').  
true.
```

```
27 ?- solve([1,2,3,4,5,6,0,7,8]).
```

```
Current State:
```

```
[1,2,3]
```

```
[4,5,6]
```

```
[0,7,8]
```

```
Current State:
```

```
[1,2,3]
```

```
[4,5,6]
```

```
[7,0,8]
```

```
Goal Reached:
```

```
[1,2,3]
```

```
[4,5,6]
```

```
[7,8,0]
```

```
true ▲
```

4. Introduction to Python Programming: Learn the different libraries - NumPy, Pandas, SciPy, Matplotlib, Scikit Learn.

Code:

```
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

```
# Step 1: Create / Load Dataset
```

```
data = {
    'Hours': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Marks': [10, 20, 30, 40, 50, 60, 70, 80, 90, 95]
}
df = pd.DataFrame(data)
print("Dataset:\n", df)
```

```
# _____
```

```
# Step 2: NumPy Operations
```

```
# _____
```

```
hours_array = np.array(df['Hours'])
marks_array = np.array(df['Marks'])
```

```
print("\nMean Marks (NumPy):", np.mean(marks_array))
print("Standard Deviation (NumPy):", np.std(marks_array))
```

```
# _____
```

```
# Step 3: SciPy Statistical Analysis
```

```
# _____
```

```
correlation, p_value = stats.pearsonr(hours_array, marks_array)
print("\nCorrelation (SciPy):", correlation)
```

```
# _____
```

```
# Step 4: Data Visualization (Matplotlib)
```

```
# _____
```

```
plt.scatter(df['Hours'], df['Marks'])
plt.title("Study Hours vs Marks")
plt.xlabel("Hours")
plt.ylabel("Marks")
plt.show()
```

```
# _____
```

```
# Step 5: Machine Learning (Scikit-learn)
```

```
# _____
```

```
X = df[['Hours']] # Independent variable
```

```
y = df['Marks'] # Dependent variable

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict
predictions = model.predict(X_test)

print("\nPredicted Marks:", predictions)
print("Actual Marks:", list(y_test))

# Model Accuracy
score = model.score(X_test, y_test)
print("\nModel Accuracy:", score)
```

If error numpy not found run :

```
!pip install numpy pandas scipy matplotlib scikit-learn
```

Output:

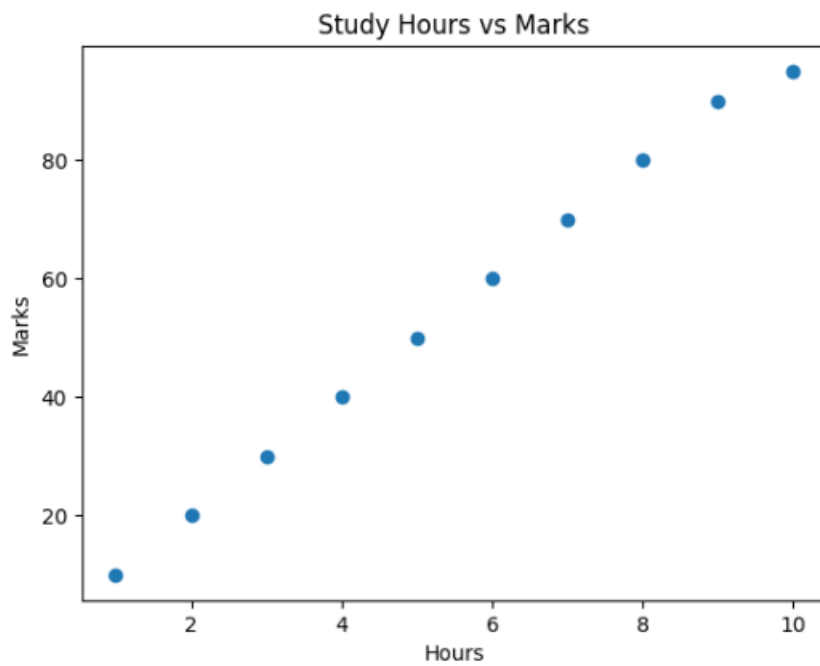
Dataset:

	Hours	Marks
0	1	10
1	2	20
2	3	30
3	4	40
4	5	50
5	6	60
6	7	70
7	8	80
8	9	90
9	10	95

Mean Marks (NumPy): 54.5

Standard Deviation (NumPy): 27.968732541893992

Correlation (SciPy): 0.9989535185064017



Predicted Marks: [60. 100.]

Actual Marks: [60, 95]

Model Accuracy: 0.9591836734693879

Q5. Implement Perceptron algorithm for OR operation

Code:

```
# Perceptron for OR operation

# Training data
inputs = [(0,0), (0,1), (1,0), (1,1)]
labels = [0, 1, 1, 1]

# Initialize weights and bias
w1, w2 = 0.0, 0.0
bias = 0.0
learning_rate = 0.1
epochs = 10

# Step activation function
def step(x):
    return 1 if x >= 0 else 0

# Training loop
for _ in range(epochs):
    for (x1, x2), label in zip(inputs, labels):
        linear_output = w1*x1 + w2*x2 + bias
        prediction = step(linear_output)
        # Update rule
        error = label - prediction
        w1 += learning_rate * error * x1
        w2 += learning_rate * error * x2
        bias += learning_rate * error

# Testing
print("Trained weights:", w1, w2)
print("Bias:", bias)

for x1, x2 in inputs:
    output = step(w1*x1 + w2*x2 + bias)
    print(f"{x1} OR {x2} = {output}")
```

Output:

```
Trained weights: 0.1 0.1
Bias: -0.1
0 OR 0 = 0
0 OR 1 = 1
1 OR 0 = 1
1 OR 1 = 1
```

|:

Q6. Improve the prediction accuracy by estimating the weight values for the training data using stochastic gradient descent. (Perceptron)

Code:

```
# Perceptron using Stochastic Gradient Descent (SGD)
```

```
import numpy as np
```

```
# Training dataset for AND gate
```

```
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
```

```
# Target outputs
```

```
y = np.array([0, 0, 0, 1])
```

```
# Initialize weights and bias
```

```
weights = np.zeros(2)
```

```
bias = 0
```

```
# Learning rate
```

```
lr = 0.1
```

```
# Number of epochs
```

```
epochs = 10
```

```
# Activation function
```

```
def activation(z):
```

```
    return 1 if z >= 0 else 0
```

```
# Training using SGD
```

```
for epoch in range(epochs):
```

```
    print(f"\nEpoch {epoch + 1}")
```

```
    for i in range(len(X)):
```

```
        # Linear combination
```

```
        z = np.dot(X[i], weights) + bias
```

```
        # Prediction
```

```
        y_pred = activation(z)
```

```
        # Error
```

```
        error = y[i] - y_pred
```

```
        # Weight update
```

```
weights = weights + lr * error * X[i]

# Bias update
bias = bias + lr * error

print(f"Input: {X[i]}, Predicted: {y_pred}, Error: {error}")

print("\nFinal Weights:", weights)
print("Final Bias:", bias)

# Testing
print("\nTesting Perceptron")

for i in range(len(X)):
    z = np.dot(X[i], weights) + bias
    prediction = activation(z)

    print(f"Input: {X[i]} -> Output: {prediction}")
```

Output:

Epoch 1
Input: [0 0], Predicted: 1, Error: -1
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 0, Error: 1

Epoch 2
Input: [0 0], Predicted: 1, Error: -1
Input: [0 1], Predicted: 1, Error: -1
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 0, Error: 1

Epoch 3
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 1, Error: -1
Input: [1 0], Predicted: 1, Error: -1
Input: [1 1], Predicted: 0, Error: 1

Epoch 4
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0

Epoch 5
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0

Epoch 6
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0

Epoch 7
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0

Epoch 8
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0

Epoch 9
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0

Epoch 10
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0

Final Weights: [0.2 0.1]
Final Bias: -0.20000000000000004

Testing Perceptron
Input: [0 0] -> Output: 0
Input: [0 1] -> Output: 0
Input: [1 0] -> Output: 0
Input: [1 1] -> Output: 1

[]: |

Q7. Implement Adaline algorithm for AND operation

Code:

```
import numpy as np

# -----
# Input dataset for AND operation
# -----
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# Target outputs
T = np.array([0, 0, 0, 1])

# -----
# Initialize weights and bias
# -----
w = np.zeros(2)
b = 0

# Learning rate
alpha = 0.1

# Number of epochs
epochs = 10

print("Initial Weights:", w)
print("Initial Bias:", b)

# -----
# Training Process
# -----
for epoch in range(epochs):

    print("\nEpoch", epoch + 1)

    for i in range(len(X)):

        x = X[i]
        target = T[i]

        # Net input
        yin = b + np.dot(w, x)

        # Error
        error = target - yin

        # Weight update
        w = w + alpha * error * x
```

```
# Bias update
b = b + alpha * error

print(f"Input: {x}, Target: {target}, Output: {yin:.2f}")
print("Updated Weights:", w)
print("Updated Bias:", round(b, 2))

# -----
# Testing Phase
# -----
print("\nTesting Adaline for AND Operation")

for i in range(len(X)):

    yin = b + np.dot(w, X[i])

    # Activation function
    output = 1 if yin >= 0.5 else 0

    print(f"Input: {X[i]} -> Output: {output}")
```

Output:

Initial Weights: [0. 0.]
Initial Bias: 0

Epoch 1

Input: [0 0], Target: 0, Output: 0.00
Updated Weights: [0. 0.]
Updated Bias: 0.0
Input: [0 1], Target: 0, Output: 0.00
Updated Weights: [0. 0.]
Updated Bias: 0.0
Input: [1 0], Target: 0, Output: 0.00
Updated Weights: [0. 0.]
Updated Bias: 0.0
Input: [1 1], Target: 1, Output: 0.00
Updated Weights: [0.1 0.1]
Updated Bias: 0.1

Epoch 2

Input: [0 0], Target: 0, Output: 0.10
Updated Weights: [0.1 0.1]
Updated Bias: 0.09
Input: [0 1], Target: 0, Output: 0.19
Updated Weights: [0.1 0.081]
Updated Bias: 0.07
Input: [1 0], Target: 0, Output: 0.17
Updated Weights: [0.0829 0.081]
Updated Bias: 0.05
Input: [1 1], Target: 1, Output: 0.22
Updated Weights: [0.16112 0.15922]
Updated Bias: 0.13

Epoch 3

Input: [0 0], Target: 0, Output: 0.13
Updated Weights: [0.16112 0.15922]
Updated Bias: 0.12
Input: [0 1], Target: 0, Output: 0.28
Updated Weights: [0.16112 0.1314072]
Updated Bias: 0.09
Input: [1 0], Target: 0, Output: 0.25
Updated Weights: [0.13589848 0.1314072]
Updated Bias: 0.07
Input: [1 1], Target: 1, Output: 0.33
Updated Weights: [0.20258054 0.19808926]
Updated Bias: 0.13

Epoch 4

Input: [0 0], Target: 0, Output: 0.13
Updated Weights: [0.20258054 0.19808926]
Updated Bias: 0.12
Input: [0 1], Target: 0, Output: 0.32
Updated Weights: [0.20258054 0.16635032]
Updated Bias: 0.09
Input: [1 0], Target: 0, Output: 0.29
Updated Weights: [0.17356637 0.16635032]
Updated Bias: 0.06
Input: [1 1], Target: 1, Output: 0.40
Updated Weights: [0.23371999 0.22650395]
Updated Bias: 0.12

Epoch 5

Input: [0 0], Target: 0, Output: 0.12
Updated Weights: [0.23371999 0.22650395]
Updated Bias: 0.11
Input: [0 1], Target: 0, Output: 0.33
Updated Weights: [0.23371999 0.19317049]
Updated Bias: 0.07
Input: [1 0], Target: 0, Output: 0.31
Updated Weights: [0.20299828 0.19317049]
Updated Bias: 0.04
Input: [1 1], Target: 1, Output: 0.44
Updated Weights: [0.25910386 0.24927607]
Updated Bias: 0.1

Epoch 6

Input: [0 0], Target: 0, Output: 0.10
Updated Weights: [0.25910386 0.24927607]
Updated Bias: 0.09
Input: [0 1], Target: 0, Output: 0.34
Updated Weights: [0.25910386 0.21544917]
Updated Bias: 0.06
Input: [1 0], Target: 0, Output: 0.31
Updated Weights: [0.22767687 0.21544917]
Updated Bias: 0.02
Input: [1 1], Target: 1, Output: 0.47
Updated Weights: [0.28099036 0.26876266]
Updated Bias: 0.08

Epoch 7

Input: [0 0], Target: 0, Output: 0.08
Updated Weights: [0.28099036 0.26876266]
Updated Bias: 0.07
Input: [0 1], Target: 0, Output: 0.34
Updated Weights: [0.28099036 0.23495167]
Updated Bias: 0.04
Input: [1 0], Target: 0, Output: 0.32
Updated Weights: [0.2493377 0.23495167]
Updated Bias: 0.0
Input: [1 1], Target: 1, Output: 0.49
Updated Weights: [0.3005204 0.28613437]
Updated Bias: 0.06

Epoch 8

Input: [0 0], Target: 0, Output: 0.06
Updated Weights: [0.3005204 0.28613437]
Updated Bias: 0.05
Input: [0 1], Target: 0, Output: 0.34
Updated Weights: [0.3005204 0.25256497]
Updated Bias: 0.02
Input: [1 0], Target: 0, Output: 0.32
Updated Weights: [0.26886933 0.25256497]
Updated Bias: -0.02
Input: [1 1], Target: 1, Output: 0.51
Updated Weights: [0.31829198 0.30198762]
Updated Bias: 0.03

Epoch 9

Input: [0 0], Target: 0, Output: 0.03
Updated Weights: [0.31829198 0.30198762]
Updated Bias: 0.03
Input: [0 1], Target: 0, Output: 0.33
Updated Weights: [0.31829198 0.26875029]
Updated Bias: -0.0
Input: [1 0], Target: 0, Output: 0.32
Updated Weights: [0.28674795 0.26875029]
Updated Bias: -0.03
Input: [1 1], Target: 1, Output: 0.52
Updated Weights: [0.33463769 0.31664003]
Updated Bias: 0.01

Epoch 10

Input: [0 0], Target: 0, Output: 0.01
Updated Weights: [0.33463769 0.31664003]
Updated Bias: 0.01
Input: [0 1], Target: 0, Output: 0.33
Updated Weights: [0.33463769 0.28376156]
Updated Bias: -0.02
Input: [1 0], Target: 0, Output: 0.31
Updated Weights: [0.30324731 0.28376156]
Updated Bias: -0.05
Input: [1 1], Target: 1, Output: 0.53
Updated Weights: [0.34975884 0.3302731]
Updated Bias: -0.01

Testing Adaline for AND Operation

Input: [0 0] -> Output: 0
Input: [0 1] -> Output: 0
Input: [1 0] -> Output: 0
Input: [1 1] -> Output: 1

Machine Learning Section

Q1. Implementation of Features Extraction and Selection, Normalization, Transformation, Principal Components Analysis

In order to run this code you need heath_data.csv file for data
Contents of heath_data.csv

ID	Age	Height (cm)	Weight (kg)	Blood Pressure (BP)	Cholesterol Level	Diabetes	Physical Activity (hours/week)	Smoking Habit	Disease Risk Score
1	56	159	77	122	High	1	1	0	59
2	69	185	77	157	Normal	0	0.53	1	48
3	46	163	93	122	Low	1	9.59	0	42.05
4	32	180	79	103	Normal	1	8.47	0	78.47
5	60	197	111	110	Normal	1	3.55	1	63.94
6	25	164	111	137	Normal	0	9.57	1	80.5
7	78	157	50	109	Normal	1	6.77	0	90.32
8	38	163	76	97	Low	1	4.83	1	61.73
9	56	172	111	96	Normal	1	4.93	0	98.05
10	75	189	52	156	Normal	1	0.83	0	60.81
11	36	170	119	106	High	0	0.92	0	63.66
12	40	165	76	122	Normal	1	6.02	1	55.48
13	28	194	58	137	High	0	5.54	1	9.1
14	28	167	111	165	Low	0	2.13	1	72.64
15	41	196	86	148	Low	0	9.46	1	54.74
16	70	173	100	175	Low	0	7.81	1	45.09
17	53	175	93	111	Low	0	1.13	0	91.05
18	57	174	73	119	High	0	9.31	0	29.8
19	41	194	108	127	Normal	1	9.74	0	52.36
20	20	190	81	140	Low	0	9.96	0	69.76
21	39	178	101	143	Normal	0	0.56	1	79.65
22	70	164	111	97	High	0	7.37	0	45.93
23	19	194	107	116	High	1	5.46	1	84.21
24	41	150	101	116	High	1	7.06	0	76.89
25	61	174	61	110	Low	0	9.69	1	6.62
26	47	156	88	119	Normal	1	6.88	1	4.59
27	55	158	51	117	Normal	0	8.37	0	62.08
28	19	173	52	153	Normal	0	8.67	0	34.74
29	77	150	105	158	High	1	8.38	0	20.91
30	38	193	108	150	Low	0	4.26	0	57.96
31	50	157	51	137	High	1	2.23	1	34.16
32	29	173	51	108	High	1	3.97	0	53.73
33	75	160	103	93	Normal	0	8.92	0	46.01
34	39	166	50	124	High	0	1.47	1	58.48
35	78	157	68	153	Normal	1	5.13	0	40.03
36	61	184	51	138	Low	0	2.33	0	69.77
37	42	184	102	106	Normal	0	5.81	0	18.01

38	66	182	93	133	Low	0	8.63	0	69.65
39	44	154	81	119	Normal	1	8.8	0	41.17
40	76	191	119	135	High	1	2.37	1	87.43
41	59	188	81	95	High	0	9.08	0	51.52
42	45	190	117	126	Low	1	5.92	0	97.31
43	77	177	104	113	Low	1	3.5	0	60.19
44	33	156	105	135	Low	1	7.08	0	22.38
45	32	158	66	142	Normal	1	4.82	0	82.18
46	79	157	87	149	Normal	1	3.78	1	34.51
47	79	161	73	152	Low	0	7.05	0	34.76
48	64	183	118	174	High	0	2.49	1	3.18
49	79	182	119	121	Normal	0	3.3	1	54.87
50	68	197	60	176	Normal	1	4.34	0	53.44
51	61	172	65	122	Normal	1	2.54	0	35.6
52	72	173	108	156	Low	0	4.05	0	89.42
53	69	186	119	107	Low	1	5.71	0	12.87
54	74	184	52	114	Low	0	7.41	1	33.01
55	20	193	69	143	Low	0	7.67	0	32.16
56	54	189	108	147	Low	0	8.23	0	9.23
57	68	171	85	156	High	1	7.44	0	48.11
58	24	176	68	135	High	1	6.81	0	68.78
59	38	184	116	113	Low	0	2.38	0	51.17
60	26	150	68	121	Normal	0	4	0	15.7
61	56	184	69	136	Low	0	4.78	1	37.73
62	35	186	101	175	High	0	0.83	1	0.26
63	21	196	82	112	High	1	5.28	0	86.83
64	42	163	89	155	Normal	1	4.36	0	8.45
65	77	152	88	116	High	0	8.02	0	59.73
66	31	150	50	91	Low	0	9.78	1	98.63
67	67	154	60	179	Normal	1	5.56	0	53.66
68	75	175	106	106	Low	1	3.23	1	92.4
69	26	163	99	122	Low	0	0.43	0	23.61
70	43	188	72	98	Normal	0	9.25	0	76
71	70	176	80	132	Low	0	9.19	0	53.13
72	19	158	91	137	High	1	2.53	0	72.05
73	37	164	56	128	Low	1	6.95	1	6.23
74	45	164	65	131	Normal	1	0.75	1	14.77
75	64	175	109	115	Normal	0	1.66	0	13.31
76	77	191	51	139	Normal	0	2.17	0	68.72
77	24	162	50	114	Low	1	2.94	1	84.44
78	61	181	97	113	Normal	1	9.96	1	74.96
79	78	188	61	102	Normal	0	6.97	1	3.05
80	25	198	118	149	Low	1	3.84	1	86.72
81	64	181	86	96	High	1	7.37	0	35.41
82	52	153	81	146	Low	0	9.15	1	39.72
83	31	179	58	125	Normal	1	9.59	0	10.49
84	34	186	68	134	Normal	0	0.58	1	73.74
85	53	172	97	109	Low	0	3.95	1	18.23

86	67	188	52	154	High	0	1.07	1	56.4
87	57	194	69	97	Normal	0	3.36	1	84.07
88	21	164	73	105	Normal	0	1.7	1	8.92
89	19	192	103	103	Normal	0	6.47	0	53.53
90	79	178	82	165	Low	0	3.88	0	23.32
91	23	185	73	176	Normal	1	2.29	1	34.29
92	71	162	85	104	High	0	2.66	1	47.4
93	59	181	87	155	Normal	0	3.6	0	35.51
94	21	156	74	121	Normal	1	2.6	1	64.88
95	71	171	67	176	High	1	4.53	1	47.96
96	46	177	115	152	Normal	1	0.32	0	58.42
97	35	151	103	175	Low	0	2.8	1	73.68
98	43	191	84	140	Low	1	4.11	1	55.77
99	61	194	110	114	Low	0	6.03	0	58.65
100	51	155	90	147	High	0	2.71	0	56.45

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, PowerTransformer,
LabelEncoder
from sklearn.feature_selection import SelectKBest, f_classif, VarianceThreshold
from sklearn.decomposition import PCA

# Load the healthcare dataset
file_path = "health_dataset.csv"
df = pd.read_csv(file_path)

# Display first few rows
display(df.head())

# Identify non-numeric columns
categorical_columns = df.select_dtypes(include=['object']).columns
print("Categorical Columns:", categorical_columns)

# Convert categorical columns using Label Encoding
label_encoders = {}

for col in categorical_columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Ensure all columns are numeric
df = df.apply(pd.to_numeric, errors='coerce') # Converts non-numeric values to NaN
df.fillna(0, inplace=True) # Replace NaNs with 0

# Separate features and target
```



```

target_column = 'Disease Risk Score'

if target_column not in df.columns:
    raise ValueError("Target column not found in dataset")

X = df.drop(columns=[target_column])
y = df[target_column]

# Check for missing values in target and encode if necessary
if y.isnull().sum() > 0:
    y.fillna(y.mode()[0], inplace=True) # Replace NaN with most frequent value

if y.dtype == 'object':
    le = LabelEncoder()
    y = le.fit_transform(y)

# Remove constant features
var_thresh = VarianceThreshold(threshold=0)
X = var_thresh.fit_transform(X)

# Feature Selection using ANOVA F-score
selector = SelectKBest(
    score_func=f_classif,
    k=min(8, X.shape[1]) # Ensure k does not exceed feature count
)

X_selected = selector.fit_transform(X, y)

print(X_selected)

selected_features = [
    col
    for col, keep in zip(
        df.drop(columns=[target_column]).columns,
        selector.get_support()
    )
    if keep
]

print("Selected Features:", selected_features)

# Normalization using Min-Max Scaling
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X_selected)

# Transformation using Power Transform (Box-Cox or Yeo-Johnson)
power_transformer = PowerTransformer(method='yeo-johnson') # Use 'box-cox' if no
negative values

X_transformed = power_transformer.fit_transform(X_normalized)

```

```

# Dimensionality Reduction using PCA
pca_components = min(5, X_transformed.shape[1]) # Ensure PCA components do not
exceed feature count

pca = PCA(n_components=pca_components)

X_pca = pca.fit_transform(X_transformed)

print("Explained Variance Ratio:", pca.explained_variance_ratio_)

# Convert processed data back to DataFrame
processed_df = pd.DataFrame(
    X_pca,
    columns=[f'PC{i+1}' for i in range(X_pca.shape[1])]
)

processed_df['Disease Risk Score'] = y.reset_index(drop=True)

# Save processed dataset
processed_df.to_csv("processed_health_dataset.csv", index=False)

print("Processed dataset saved successfully.")

```

Output:

	ID	Age	Height (cm)	Weight (kg)	Blood Pressure (BP)	Cholesterol Level	Diabetes	Physical Activity (hours/week)	Smoking Habit	Disease Risk Score
0	1	56	159	77	122	High	1	1.00	0	59.00
1	2	69	185	77	157	Normal	0	0.53	1	48.00
2	3	46	163	93	122	Low	1	9.59	0	42.05
3	4	32	180	79	103	Normal	1	8.47	0	78.47
4	5	60	197	111	110	Normal	1	3.55	1	63.94

Categorical Columns: Index(['Cholesterol Level'], dtype='str')

```

[[ 56. 159.  77. 122.  0.  1.  1.  0. ]
 [ 69. 185.  77. 157.  2.  0.  0.53  1. ]
 [ 46. 163.  93. 122.  1.  1.  9.59  0. ]
 [ 32. 180.  79. 103.  2.  1.  8.47  0. ]
 [ 60. 197. 111. 110.  2.  1.  3.55  1. ]
 [ 25. 164. 111. 137.  2.  0.  9.57  1. ]
 [ 78. 157.  50. 109.  2.  1.  6.77  0. ]
 [ 38. 163.  76.  97.  1.  1.  4.83  1. ]
 [ 56. 172. 111.  96.  2.  1.  4.93  0. ]
 [ 75. 189.  52. 156.  2.  1.  0.83  0. ]
 [ 36. 170. 119. 106.  0.  0.  0.92  0. ]
 [ 40. 165.  76. 122.  2.  1.  6.02  1. ]
 [ 28. 194.  58. 137.  0.  0.  5.54  1. ]
 [ 28. 167. 111. 165.  1.  0.  2.13  1. ]
 [ 41. 196.  86. 148.  1.  0.  9.46  1. ]
 [ 70. 173. 100. 175.  1.  0.  7.81  1. ]
 [ 53. 175.  93. 111.  1.  0.  1.13  0. ]
 [ 57. 174.  73. 119.  0.  0.  9.31  0. ]
 [ 41. 194. 108. 127.  2.  1.  9.74  0. ]
 [ 20. 190.  81. 140.  1.  0.  9.96  0. ]
 [ 39. 178. 101. 143.  2.  0.  0.56  1. ]
 [ 70. 164. 111.  97.  0.  0.  7.37  0. ]
 [ 19. 194. 107. 116.  0.  1.  5.46  1. ]
 [ 41. 150. 101. 116.  0.  1.  7.06  0. ]
 [ 61. 174.  61. 110.  1.  0.  9.69  1. ]

```

Q2. Implementation of Logistic regression

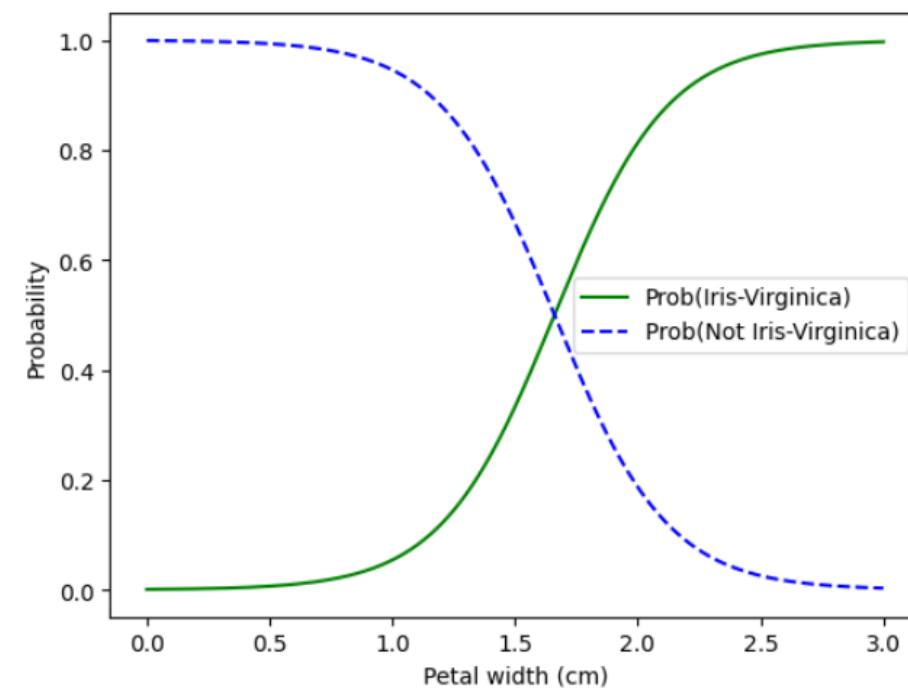
Code:

```
import numpy as np
from sklearn import datasets
iris = datasets.load_iris()
print(type(iris))
print(list(iris.keys()))
X = iris["data"][:,3:] # petal width
y = (iris["target"] == 2).astype(np.int64) # 1 if Iris-Virginica, else 0
from sklearn.linear_model import LogisticRegression
log_reg = LogisticRegression(solver="lbfgs", random_state=42)
log_reg.fit(X,y)
import matplotlib.pyplot as plt
```

```
X_new = np.linspace(0,3,1000).reshape(-1,1)
y_proba = log_reg.predict_proba(X_new)
plt.plot(X_new, y_proba[:,1], "g-")
plt.plot(X_new, y_proba[:,0], "b--")
plt.xlabel('Petal width (cm)')
plt.ylabel('Probability')
plt.legend(['Prob(Iris-Virginica)', 'Prob(Not Iris-Virginica)'])
print(log_reg.predict(X))
print(log_reg.coef_)
print(log_reg.intercept_)
```

Output:

```
<class 'sklearn.utils._bunch.Bunch'>  
['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module']  
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1  
 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 0 1 1 1 0 0 1 1 1 1 1 1  
 1 1]  
[[4.33251974]]  
[-7.19342106]
```



2. Logistic regression with multiple feature (Petal Width & Petal Height)

Code:

```
from sklearn.linear_model import LogisticRegression
X = iris["data"][:, (2, 3)] # petal length, petal width
y = (iris["target"] == 2).astype(np.int64)
log_reg2 = LogisticRegression(solver="lbfgs", C=10**10, random_state=42)
log_reg2.fit(X, y)
x0, x1 = np.meshgrid(
    np.linspace(2.9, 7, 500).reshape(-1, 1),
    np.linspace(0.8, 2.7, 200).reshape(-1, 1),
)
```

```

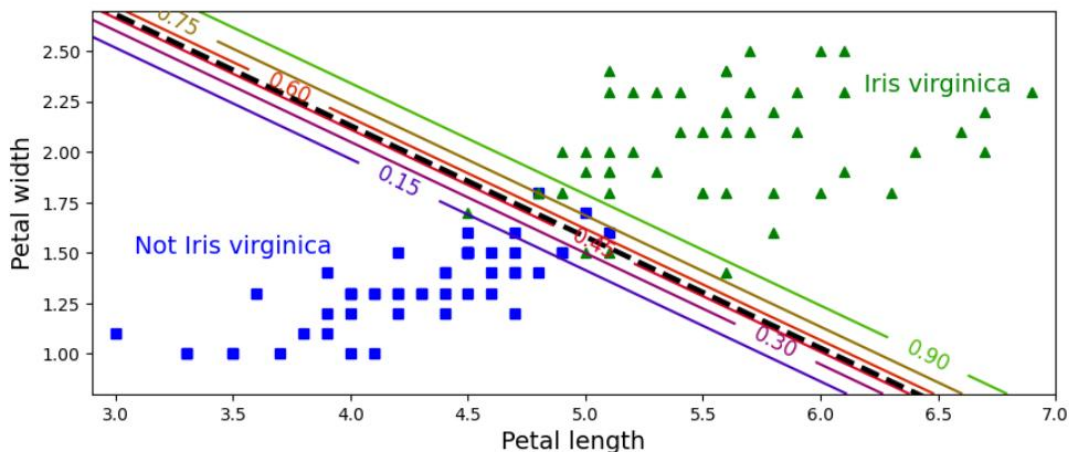
X_new = np.c_[x0.ravel(), x1.ravel()]
print(X_new.shape)
y_proba = log_reg2.predict_proba(X_new)
plt.figure(figsize=(10, 4))
plt.plot(X[y==0, 0], X[y==0, 1], "bs")
plt.plot(X[y==1, 0], X[y==1, 1], "g^")
zz = y_proba[:, 1].reshape(x0.shape)
contour = plt.contour(x0, x1, zz, cmap=plt.cm.brg)
left_right = np.array([2.9, 7])
boundary = -(log_reg2.coef_[0][0] * left_right + log_reg2.intercept_[0]) / log_reg2.coef_[0][1]
plt.clabel(contour, inline=1, fontsize=12)
plt.plot(left_right, boundary, "k--", linewidth=3)
plt.text(3.5, 1.5, "Not Iris virginica", fontsize=14, color="b", ha="center")
plt.text(6.5, 2.3, "Iris virginica", fontsize=14, color="g", ha="center")
plt.xlabel("Petal length", fontsize=14)
plt.ylabel("Petal width", fontsize=14)
plt.axis([2.9, 7, 0.8, 2.7])

```

Output:

(100000, 2)

(np.float64(2.9), np.float64(7.0), np.float64(0.8), np.float64(2.7))



Q3. Implementation of Classifying data using Support Vector Machine (SVM)- Linear and Non-Linear SVM Classification

In order to run this code you need heath_data.csv file for data

Contents of heath_data.csv

ID	Age	Height (cm)	Weight (kg)	Blood Pressure (BP)	Cholesterol Level	Diabetes	Physical Activity (hours/week)	Smoking Habit	Disease Risk Score
1	56	159	77	122	High	1	1	0	59
2	69	185	77	157	Normal	0	0.53	1	48
3	46	163	93	122	Low	1	9.59	0	42.05
4	32	180	79	103	Normal	1	8.47	0	78.47
5	60	197	111	110	Normal	1	3.55	1	63.94
6	25	164	111	137	Normal	0	9.57	1	80.5
7	78	157	50	109	Normal	1	6.77	0	90.32
8	38	163	76	97	Low	1	4.83	1	61.73
9	56	172	111	96	Normal	1	4.93	0	98.05
10	75	189	52	156	Normal	1	0.83	0	60.81
11	36	170	119	106	High	0	0.92	0	63.66
12	40	165	76	122	Normal	1	6.02	1	55.48
13	28	194	58	137	High	0	5.54	1	9.1
14	28	167	111	165	Low	0	2.13	1	72.64
15	41	196	86	148	Low	0	9.46	1	54.74
16	70	173	100	175	Low	0	7.81	1	45.09
17	53	175	93	111	Low	0	1.13	0	91.05
18	57	174	73	119	High	0	9.31	0	29.8
19	41	194	108	127	Normal	1	9.74	0	52.36
20	20	190	81	140	Low	0	9.96	0	69.76
21	39	178	101	143	Normal	0	0.56	1	79.65
22	70	164	111	97	High	0	7.37	0	45.93
23	19	194	107	116	High	1	5.46	1	84.21
24	41	150	101	116	High	1	7.06	0	76.89
25	61	174	61	110	Low	0	9.69	1	6.62
26	47	156	88	119	Normal	1	6.88	1	4.59
27	55	158	51	117	Normal	0	8.37	0	62.08
28	19	173	52	153	Normal	0	8.67	0	34.74

29	77	150	105	158	High	1	8.38	0	20.91
30	38	193	108	150	Low	0	4.26	0	57.96
31	50	157	51	137	High	1	2.23	1	34.16
32	29	173	51	108	High	1	3.97	0	53.73
33	75	160	103	93	Normal	0	8.92	0	46.01
34	39	166	50	124	High	0	1.47	1	58.48
35	78	157	68	153	Normal	1	5.13	0	40.03
36	61	184	51	138	Low	0	2.33	0	69.77
37	42	184	102	106	Normal	0	5.81	0	18.01
38	66	182	93	133	Low	0	8.63	0	69.65
39	44	154	81	119	Normal	1	8.8	0	41.17
40	76	191	119	135	High	1	2.37	1	87.43
41	59	188	81	95	High	0	9.08	0	51.52
42	45	190	117	126	Low	1	5.92	0	97.31
43	77	177	104	113	Low	1	3.5	0	60.19
44	33	156	105	135	Low	1	7.08	0	22.38
45	32	158	66	142	Normal	1	4.82	0	82.18
46	79	157	87	149	Normal	1	3.78	1	34.51
47	79	161	73	152	Low	0	7.05	0	34.76
48	64	183	118	174	High	0	2.49	1	3.18
49	79	182	119	121	Normal	0	3.3	1	54.87
50	68	197	60	176	Normal	1	4.34	0	53.44
51	61	172	65	122	Normal	1	2.54	0	35.6
52	72	173	108	156	Low	0	4.05	0	89.42
53	69	186	119	107	Low	1	5.71	0	12.87
54	74	184	52	114	Low	0	7.41	1	33.01
55	20	193	69	143	Low	0	7.67	0	32.16
56	54	189	108	147	Low	0	8.23	0	9.23
57	68	171	85	156	High	1	7.44	0	48.11
58	24	176	68	135	High	1	6.81	0	68.78
59	38	184	116	113	Low	0	2.38	0	51.17
60	26	150	68	121	Normal	0	4	0	15.7
61	56	184	69	136	Low	0	4.78	1	37.73
62	35	186	101	175	High	0	0.83	1	0.26
63	21	196	82	112	High	1	5.28	0	86.83
64	42	163	89	155	Normal	1	4.36	0	8.45
65	77	152	88	116	High	0	8.02	0	59.73
66	31	150	50	91	Low	0	9.78	1	98.63
67	67	154	60	179	Normal	1	5.56	0	53.66
68	75	175	106	106	Low	1	3.23	1	92.4
69	26	163	99	122	Low	0	0.43	0	23.61
70	43	188	72	98	Normal	0	9.25	0	76
71	70	176	80	132	Low	0	9.19	0	53.13
72	19	158	91	137	High	1	2.53	0	72.05
73	37	164	56	128	Low	1	6.95	1	6.23
74	45	164	65	131	Normal	1	0.75	1	14.77
75	64	175	109	115	Normal	0	1.66	0	13.31
76	77	191	51	139	Normal	0	2.17	0	68.72

77	24	162	50	114	Low	1	2.94	1	84.44
78	61	181	97	113	Normal	1	9.96	1	74.96
79	78	188	61	102	Normal	0	6.97	1	3.05
80	25	198	118	149	Low	1	3.84	1	86.72
81	64	181	86	96	High	1	7.37	0	35.41
82	52	153	81	146	Low	0	9.15	1	39.72
83	31	179	58	125	Normal	1	9.59	0	10.49
84	34	186	68	134	Normal	0	0.58	1	73.74
85	53	172	97	109	Low	0	3.95	1	18.23
86	67	188	52	154	High	0	1.07	1	56.4
87	57	194	69	97	Normal	0	3.36	1	84.07
88	21	164	73	105	Normal	0	1.7	1	8.92
89	19	192	103	103	Normal	0	6.47	0	53.53
90	79	178	82	165	Low	0	3.88	0	23.32
91	23	185	73	176	Normal	1	2.29	1	34.29
92	71	162	85	104	High	0	2.66	1	47.4
93	59	181	87	155	Normal	0	3.6	0	35.51
94	21	156	74	121	Normal	1	2.6	1	64.88
95	71	171	67	176	High	1	4.53	1	47.96
96	46	177	115	152	Normal	1	0.32	0	58.42
97	35	151	103	175	Low	0	2.8	1	73.68
98	43	191	84	140	Low	1	4.11	1	55.77
99	61	194	110	114	Low	0	6.03	0	58.65
100	51	155	90	147	High	0	2.71	0	56.45

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, PowerTransformer,
LabelEncoder
from sklearn.feature_selection import SelectKBest, f_classif, VarianceThreshold
from sklearn.decomposition import PCA

# Load the healthcare dataset
file_path = "health_dataset.csv"
df = pd.read_csv(file_path)

# Display first few rows
display(df.head())

# Identify non-numeric columns
categorical_columns = df.select_dtypes(include=['object']).columns
print("Categorical Columns:", categorical_columns)

# Convert categorical columns using Label Encoding
label_encoders = {}
```



```

for col in categorical_columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Ensure all columns are numeric
df = df.apply(pd.to_numeric, errors='coerce') # Converts non-numeric values to NaN
df.fillna(0, inplace=True)                  # Replace NaNs with 0

# Separate features and target
target_column = 'Disease Risk Score'

if target_column not in df.columns:
    raise ValueError("Target column not found in dataset")

X = df.drop(columns=[target_column])
y = df[target_column]

# Check for missing values in target and encode if necessary
if y.isnull().sum() > 0:
    y.fillna(y.mode()[0], inplace=True) # Replace NaN with most frequent value

if y.dtype == 'object':
    le = LabelEncoder()
    y = le.fit_transform(y)

# Remove constant features
var_thresh = VarianceThreshold(threshold=0)
X = var_thresh.fit_transform(X)

# Feature Selection using ANOVA F-score
selector = SelectKBest(
    score_func=f_classif,
    k=min(8, X.shape[1]) # Ensure k does not exceed feature count
)

X_selected = selector.fit_transform(X, y)

print(X_selected)

selected_features = [
    col
    for col, keep in zip(
        df.drop(columns=[target_column]).columns,
        selector.get_support()
    )
    if keep
]

print("Selected Features:", selected_features)

```

```
# Normalization using Min-Max Scaling
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X_selected)

# Transformation using Power Transform (Box-Cox or Yeo-Johnson)
power_transformer = PowerTransformer(method='yeo-johnson') # Use 'box-cox' if no
negative values

X_transformed = power_transformer.fit_transform(X_normalized)

# Dimensionality Reduction using PCA
pca_components = min(5, X_transformed.shape[1]) # Ensure PCA components do not
exceed feature count

pca = PCA(n_components=pca_components)

X_pca = pca.fit_transform(X_transformed)

print("Explained Variance Ratio:", pca.explained_variance_ratio_)

# Convert processed data back to DataFrame
processed_df = pd.DataFrame(
    X_pca,
    columns=[f'PC{i+1}' for i in range(X_pca.shape[1])]
)

processed_df['Disease Risk Score'] = y.reset_index(drop=True)

# Save processed dataset
processed_df.to_csv("processed_health_dataset.csv", index=False)

print("Processed dataset saved successfully.")
```

Output:

	ID	Age	Height (cm)	Weight (kg)	Blood Pressure (BP)	Cholesterol Level	Diabetes	Physical Activity (hours/week)	Smoking Habit	Disease Risk Score
0	1	56	159	77	122	High	1	1.00	0	59.00
1	2	69	185	77	157	Normal	0	0.53	1	48.00
2	3	46	163	93	122	Low	1	9.59	0	42.05
3	4	32	180	79	103	Normal	1	8.47	0	78.47
4	5	60	197	111	110	Normal	1	3.55	1	63.94
Categorical Columns: Index(['Cholesterol Level'], dtype='str')										
[	56.	159.	77.	122.	0.	1.	1.	0.	]	
[	69.	185.	77.	157.	2.	0.	0.53	1.	]	
[	46.	163.	93.	122.	1.	1.	9.59	0.	]	
[	32.	180.	79.	103.	2.	1.	8.47	0.	]	
[	60.	197.	111.	110.	2.	1.	3.55	1.	]	
[	25.	164.	111.	137.	2.	0.	9.57	1.	]	
[	78.	157.	50.	109.	2.	1.	6.77	0.	]	
[	38.	163.	76.	97.	1.	1.	4.83	1.	]	
[	56.	172.	111.	96.	2.	1.	4.93	0.	]	
[	75.	189.	52.	156.	2.	1.	0.83	0.	]	
[	36.	170.	119.	106.	0.	0.	0.92	0.	]	
[	40.	165.	76.	122.	2.	1.	6.02	1.	]	
[	28.	194.	58.	137.	0.	0.	5.54	1.	]	
[	28.	167.	111.	165.	1.	0.	2.13	1.	]	
[	41.	196.	86.	148.	1.	0.	9.46	1.	]	
[	70.	173.	100.	175.	1.	0.	7.81	1.	]	
[	53.	175.	93.	111.	1.	0.	1.13	0.	]	
[	57.	174.	73.	119.	0.	0.	9.31	0.	]	
[	41.	194.	108.	127.	2.	1.	9.74	0.	]	
[	20.	190.	81.	140.	1.	0.	9.96	0.	]	
[	39.	178.	101.	143.	2.	0.	0.56	1.	]	
[	70.	164.	111.	97.	0.	0.	7.37	0.	]	
[	19.	194.	107.	116.	0.	1.	5.46	1.	]	
[	41.	150.	101.	116.	0.	1.	7.06	0.	]	
[	61.	174.	61.	110.	1.	0.	9.69	1.	]	

localhost:8888/edit/AIML_Practicals/processed_health_dataset.csv

jupyter
processed_health_dataset.csv
Last Checkpoint: 6 minutes ago

File
Edit
View
Settings
Help

Delimiter:						
	PC1	PC2	PC3	PC4	PC5	Disease Risk Score
1	-0.014877293300707407	-0.6984988464326285	1.8149276838307873	-0.8521416161664792	-0.45045181735032086	59.0
2	-2.359992516649972	-0.09345991649235831	-0.3201249144584501	1.7180333266053127	0.5851405437284323	48.0
3	1.7974557524439234	-0.23579985115004726	0.7013296350823274	-0.8229933209357106	0.17721701692767244	42.05
4	1.8989990004703765	-0.46830660179277933	-1.2114071470491716	-0.6680976500493176	1.0772913060670353	78.47
5	-0.6937586613192201	0.6331666020612142	-0.9409772088350431	-0.2928729081568294	1.761311771592179	63.94
6	0.10824914208926387	0.0022322037406680055	-1.2421888638198084	-0.4656282346266701	-0.1155820737075183	80.5
7	2.048268265359671	-1.6155957129939833	0.5673155673967925	1.4979293679037684	0.316731512391377	90.32
8	0.49152803550524776	-1.2242163842862812	-0.6247522939903075	-1.1330235346290067	-0.7162962852903844	61.73
9	1.4823599248700339	0.31134778519165884	-0.07742145087985196	-0.5939754998859116	1.5398568567614712	98.05
10	-0.87412483012903	-1.0098861016830671	0.6786172144060166	1.6294579227716837	1.7934373743502652	60.81
11	-0.35530902547909904	1.563565252440689	0.42145164576962213	-1.6594132974167444	-0.6461869530422605	63.66
12	0.19187272662712598	-1.6526374677519575	-0.7051141977014834	-0.2617484090096843	0.37730389567927847	55.48
13	-1.2821373632959057	0.24920796464709283	-1.3154217650825006	-0.34787125356091736	-1.6446575195843567	9.1
14	-2.1349272623984152	0.07944522108408293	0.06223251566298515	-0.8909182111001939	-0.34058684303951564	72.64
15	-0.6935701510037348	0.9949307747396532	-1.2578474513607527	0.21228821787131005	-0.4763511091142666	54.74
16	-1.0050530718179005	0.7181674731006797	0.72764454628156	1.0304316323702332	-0.5056201051748338	45.09
17	-0.26945010036226047	0.9110117113811949	0.07675849625017127	-0.023461554128295974	-0.08628049917724186	91.05
18	1.2877231327144565	1.0098802825469502	0.3020889141497837	0.3053371752497895	-1.5320924108151492	29.8
19	1.2220433401372828	0.781469031056776	-0.6813746794231479	-0.6240307342928009	2.0387672920441235	52.36
20	0.6531844849214307	0.8857164605809354	-1.200580689659328	-0.5015922810844515	-0.22244085086463422	69.76
21	-2.146682181147548	-0.002309040660430796	-0.8656853094835407	0.0838653155607396	0.6230473731402464	79.65
22	1.5785918395998928	1.9177125702616185	0.872011742195046	-0.23811827546433387	-1.2717914942572937	45.93
23	-0.7217019468342745	0.6651808331966397	-0.8539992026275632	-2.6091809280907334	-0.1477566059975248	84.21
24	1.521833336767275	-0.2619622444268823	1.5090169168080703	-1.7658460868560835	-0.7128231439758819	76.89
25	0.7113160815391487	0.03316302340818778	-1.0256932737745086	1.072700767320472	-1.6574719876766517	6.62
26	0.5764994345109647	-1.507348184638997	-0.2438014980560841	-0.2711189833048094	0.29909454134900426	4.59
27	1.7618167252949226	-0.9219699623127966	-0.5520961687964808	1.596076106635936	-0.6200152688039072	62.08

Q4. Implement Elbow method for K means Clustering

Code:

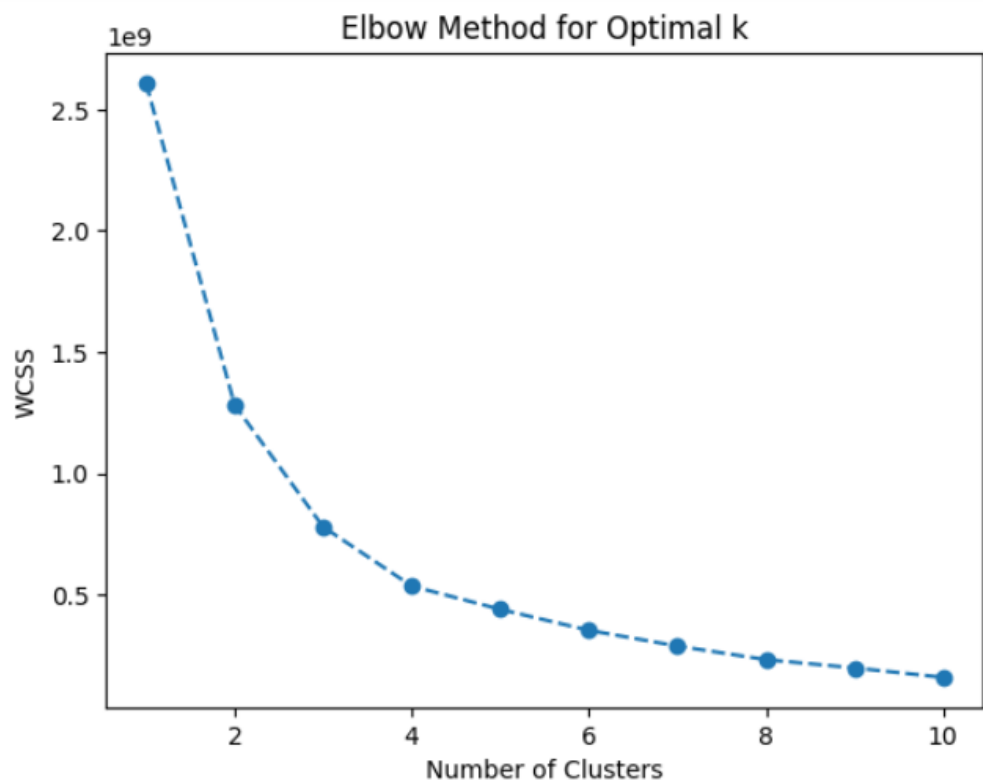
Output:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001003	Male	Yes	1	Graduate	No	
1	LP001005	Male	Yes	0	Graduate	Yes	
2	LP001006	Male	Yes	0	Not Graduate	No	
3	LP001008	Male	No	0	Graduate	No	
4	LP001013	Male	Yes	0	Not Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	4583	1508.0	128.0	360.0	
1	3000	0.0	66.0	360.0	
2	2583	2358.0	120.0	360.0	
3	6000	0.0	141.0	360.0	
4	2333	1516.0	95.0	360.0	

	Credit_History	Property_Area	Loan_Status
0	1.0	Rural	N
1	1.0	Urban	Y
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

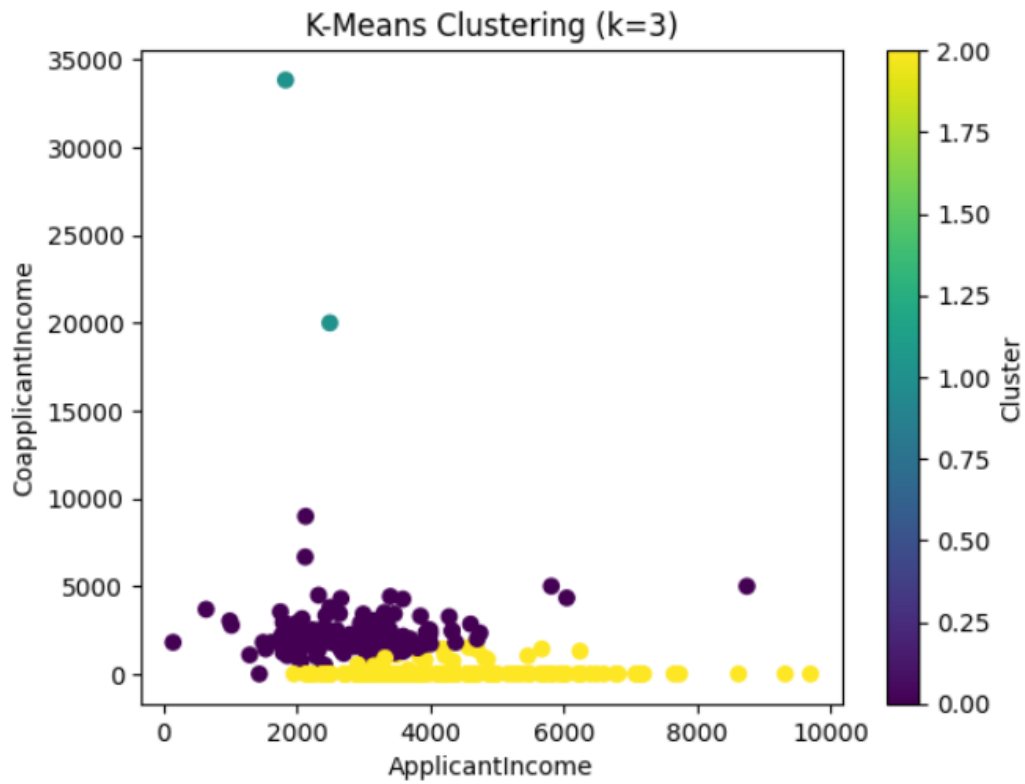
Numerical columns used for clustering: ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Loan_Amount_Term', 'Credit_History']



	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001003	Male	Yes	1	Graduate	No	
1	LP001005	Male	Yes	0	Graduate	Yes	
2	LP001006	Male	Yes	0	Not Graduate	No	
3	LP001008	Male	No	0	Graduate	No	
4	LP001013	Male	Yes	0	Not Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	4583	1508.0	128.0	360.0	
1	3000	0.0	66.0	360.0	
2	2583	2358.0	120.0	360.0	
3	6000	0.0	141.0	360.0	
4	2333	1516.0	95.0	360.0	

	Credit_History	Property_Area	Loan_Status	Cluster
0	1.0	Rural	N	2
1	1.0	Urban	Y	2
2	1.0	Urban	Y	0
3	1.0	Urban	Y	2
4	1.0	Urban	Y	0



Q5. Implementation of Bagging Algorithm: Random Forest

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.decomposition import PCA
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Initialize and train the Random Forest Classifier
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X_train, y_train)
# Make predictions
y_pred = rf_classifier.predict(X_test)
# Evaluate the model
cm=confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy of Random Forest Classifier: {accuracy * 100:.2f}%')
print("Confusion Matrix:\n", cm)
```

Output:

```
Accuracy of Random Forest Classifier: 100.00%
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Q6. Implementation of Boosting Algorithms: AdaBoost, Stochastic Gradient Boosting, Voting Ensemble

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.decomposition import PCA
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Initialize and train the Random Forest Classifier
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X_train, y_train)
# Make predictions
y_pred = rf_classifier.predict(X_test)
# Evaluate the model
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy of Random Forest Classifier: {accuracy * 100:.2f}%')
print("Confusion Matrix:\n", cm)
```

Output:

```
Accuracy of Random Forest Classifier: 100.00%
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

1. |